

Practical 1

Write a program to print numbers in words using Nested if and Switch Case.

Theory

Decision-making is an important concept in programming. It allows a program to choose different actions based on conditions. Java provides control structures like **if-else** and **switch case** for this purpose.

A **nested if** statement is an if statement inside another if or else block. It allows checking multiple conditions in a structured way.

A **switch case** statement is used when a variable needs to be compared with multiple constant values. It is often used when there are many possible values for a single variable.

In this program, a number between 1 and 5 is entered, and the program prints its equivalent word form such as "One", "Two", etc. The nested if ensures the number is within range, and the switch statement converts it into words.

Code

```
import java.util.Scanner;

public class NumberInWords {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter number (1-5): ");
        int num = sc.nextInt();

        if (num >= 1 && num <= 5) {
            switch (num) {
                case 1: System.out.println("One"); break;
                case 2: System.out.println("Two"); break;
                case 3: System.out.println("Three"); break;
                case 4: System.out.println("Four"); break;
                case 5: System.out.println("Five"); break;
            }
        } else {
```

```
        System.out.println("Number out of range");
    }
}
}
```

Output

```
Enter number (1-5): 3
Three
```

Practical 2

PassFail program (print PASS if marks $\geq$ 50 else FAIL)

Theory

Conditional statements allow a program to make decisions. The **if-else** statement is used when there are two possible outcomes.

In education systems, students are evaluated based on marks. If the marks are above a certain threshold, the student passes; otherwise, the student fails.

This program demonstrates how to use the if-else statement in Java to check whether a student has passed or failed based on marks entered.

Code

```
import java.util.Scanner;

public class PassFail {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter marks: ");
        int marks = sc.nextInt();

        if (marks >= 50)
            System.out.println("PASS");
    }
}
```

```
        else
            System.out.println("FAIL");
    }
}
```

Output

```
Enter marks: 45
FAIL
```

Practical 3

OddEven program

Theory

Numbers can be classified as even or odd.

- Even numbers are divisible by 2.
- Odd numbers are not divisible by 2.

In programming, the modulus operator `%` is used to find the remainder. If a number divided by 2 gives remainder 0, it is even.

This program demonstrates how to use conditional statements and modulus operator to check whether a number is odd or even.

Code

```
import java.util.Scanner;

public class OddEven {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter number: ");
        int num = sc.nextInt();
```

```
        if (num % 2 == 0)
            System.out.println("Even Number");
        else
            System.out.println("Odd Number");
    }
}
```

Output

```
Enter number: 7
Odd Number
```

Practical 4

Find sum and average of 10 numbers using array.

Theory

An array is a collection of elements of the same data type stored in contiguous memory locations. Arrays are useful when we need to store multiple values under a single variable name.

In this program, an array is used to store 10 numbers. A loop is used to calculate the sum of all elements. The average is then calculated by dividing the sum by the total number of elements.

This demonstrates the use of arrays and loops in Java.

Code

```
import java.util.Scanner;

public class SumAverage {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int[] arr = new int[10];
        int sum = 0;
```

```
        System.out.println("Enter 10 numbers:");
        for (int i = 0; i < 10; i++) {
            arr[i] = sc.nextInt();
            sum += arr[i];
        }

        double avg = sum / 10.0;

        System.out.println("Sum = " + sum);
        System.out.println("Average = " + avg);
    }
}
```

Output

```
Enter 10 numbers:
1 2 3 4 5 6 7 8 9 10
Sum = 55
Average = 5.5
```

Practical 5

Reverse of a digit using array.

Theory

Reversing digits of a number is a common programming task. It involves extracting each digit and storing it in reverse order.

An array can be used to store the digits, and then they can be printed in reverse order. This demonstrates the use of arrays and loops.

Code

```
import java.util.Scanner;
```

```
public class ReverseDigits {  
    public static void main(String[] args) {  
        Scanner sc = new Scanner(System.in);  
        System.out.print("Enter number: ");  
        int num = sc.nextInt();  
  
        int[] digits = new int[10];  
        int i = 0;  
  
        while (num > 0) {  
            digits[i++] = num % 10;  
            num /= 10;  
        }  
  
        System.out.print("Reversed number: ");  
        for (int j = 0; j < i; j++) {  
            System.out.print(digits[j]);  
        }  
    }  
}
```

Output

```
Enter number: 1234  
Reversed number: 4321
```

Practical 6

Display grade according to marks.

Theory

Grading systems classify student performance into categories. Conditional statements are used to check the marks range and assign the appropriate grade.

This program demonstrates how multiple conditions can be checked using if–else if statements.

Code

```
import java.util.Scanner;

public class Grade {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter marks: ");
        int marks = sc.nextInt();

        if (marks >= 90)
            System.out.println("Grade A");
        else if (marks >= 75)
            System.out.println("Grade B");
        else if (marks >= 60)
            System.out.println("Grade C");
        else
            System.out.println("Fail");
    }
}
```

Output

```
Enter marks: 82
Grade B
```

Practical 7

Find factorial using command line argument.

Theory

Command-line arguments allow data to be passed to a program when it is executed. In Java, these arguments are stored in the `args[]` array of the main method.

Factorial is the product of all integers from 1 to the given number.

This program demonstrates how to read input from command-line arguments and compute factorial.

Code

```
public class FactorialCLI {  
    public static void main(String[] args) {  
        int num = Integer.parseInt(args[0]);  
        int fact = 1;  
  
        for (int i = 1; i <= num; i++) {  
            fact *= i;  
        }  
  
        System.out.println("Factorial = " + fact);  
    }  
}
```

Output

Command:

```
java FactorialCLI 5
```

Output:

```
Factorial = 120
```

Practical 8

Print Fibonacci series.

Theory

The Fibonacci series is a sequence where each number is the sum of the two preceding numbers. The series starts with 0 and 1.

It is commonly used in mathematics and computer science to demonstrate loops and recursion.

Code

```
import java.util.Scanner;

public class Fibonacci {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter number of terms: ");
        int n = sc.nextInt();

        int a = 0, b = 1;

        System.out.print("Fibonacci series: ");
        for (int i = 1; i <= n; i++) {
            System.out.print(a + " ");
            int c = a + b;
            a = b;
            b = c;
        }
    }
}
```

Output

```
Enter number of terms: 5
Fibonacci series: 0 1 1 2 3
```

Practical 9

Display tables from 2 to 10.

Theory

Loops are used to repeat a block of code multiple times. Nested loops are used when one loop is inside another loop.

In this program, a nested loop is used to generate multiplication tables from 2 to 10.

Code

```
public class Tables {  
    public static void main(String[] args) {  
        for (int i = 2; i <= 10; i++) {  
            System.out.println("Table of " + i);  
            for (int j = 1; j <= 10; j++) {  
                System.out.println(i + " x " + j + " = " + (i * j));  
            }  
            System.out.println();  
        }  
    }  
}
```

Output (partial)

```
Table of 2  
2 x 1 = 2  
2 x 2 = 4  
...  
2 x 10 = 20
```

Practical 10

Check whether a number is prime.

Theory

A prime number is a number greater than 1 that has only two factors: 1 and itself. Examples include 2, 3, 5, 7, and 11.

To check whether a number is prime, we test whether it is divisible by any number other than 1 and itself.

This program demonstrates how to check for prime numbers using loops and conditions.

Code

```
import java.util.Scanner;

public class PrimeCheck {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter number: ");
        int num = sc.nextInt();
        boolean isPrime = true;

        if (num <= 1)
            isPrime = false;
        else {
            for (int i = 2; i <= num / 2; i++) {
                if (num % i == 0) {
                    isPrime = false;
                    break;
                }
            }
        }

        if (isPrime)
            System.out.println("Prime number");
        else
            System.out.println("Not a prime number");
    }
}
```

Output

```
Enter number: 7
Prime number
```